Super-Precise Latency: How CPU Threading Affects High-Precision Latency, and an Industry-Wide Audit

Gustavo Pedro Ricou and Romaric Duvignau

***Abstract*—Engineers choose message brokers, which carry data between programs, partly by latency benchmarks that now report latencies below a millisecond. At that scale, when a program reads the clock matters. After a broker writes a message, it acknowledges it to the producer and delivers it to the consumer. On one machine and one clock, the two readings should coincide. Measured, their difference spreads over milliseconds, and part of it falls below zero. The cause is scheduling. A thread must wait for a processor core before reading the clock, and the thread that reads the acknowledgment can wait longer than the message takes to reach the consumer. How often this happens obeys laws we derive and test by manipulation. It rises with load, depends on how the load is placed, and falls when the clock-reading threads run at real-time priority and as the timed path outgrows the scheduler's slice, which the processor count sets. An industry-wide audit finds such values discarded uncounted or never measured, and a filter on millisecond clocks that deletes the fastest messages too. Kernel timestamps, readable by any tool, exclude the wait, even under load. We give checks, and measure what most of them buy.**

***Index Terms*—Apache Kafka; benchmark auditing; CPU scheduling; latency benchmarking; message brokers; multi-threading; Redis Streams; timestamp resolution.**

I. Introduction

MANY online services move their data through a message broker, a service that takes each message from the program that publishes it, the *producer*, and delivers it to the program that receives it, the *consumer* (Fig. 1a). Engineers choose among brokers partly by latency benchmarks [1], which time each message from producer to consumer and now report latencies below a millisecond [2]. At that scale, a thread's wait for a processor core can exceed the latency itself. A team that ranks brokers or promises a response time on such a number may be acting on how the number was measured, not on the broker.

We study one difference that, in the ideal case, is zero. Once the broker has written a message, it returns an acknowledgment to the producer and delivers the message to the consumer. Both leave after the same write. When the producer and the consumer run on one machine and read its one clock, the producer's reading of the acknowledgment, t_{ack}, and the consumer's reading of the message, t_{recv}, should coincide. We make four contributions.

1) **The distribution** (Section III). On one clock, $S = t_{\mathrm{recv}} - t_{\mathrm{ack}}$ is spread over milliseconds, and 8.79% of it falls below zero. Nothing arrived early: no latency timed from the *publish call*, the moment the producer hands the message over, falls below zero.
2) **The mechanism, and its laws** (Section IV). A thread must wait for a core before it can read the clock, and the thread that reads the acknowledgment can wait longer than the message takes to reach the consumer. How often that happens rises with load and depends on where the load is placed, falls as the timed path lengthens, and has the timescale of the scheduler's slice, which the kernel sets from the CPU count. *Real-time priority*, which runs the threads that read the clock ahead of every thread at the default *normal priority*, cut it 7–$80\times$ on Kafka and 6.0–$7.8\times$ on Redis at unchanged load.
3) **The industry-wide audit** (Section V). Of the 24 message tools we read at source, 19 cut such values out of a figure they report. All but one, the Storage Benchmark Kit (SBK), count none of them. Of the rest, two never meet such a value and three keep it. Of the 32 we ran, 19 take both timestamps in one process, so they never measure a *one-way latency*, the time from one process to another. On a millisecond clock, a filter that keeps only latencies above zero, a *positivity filter*, also deletes the fastest messages, and no public report we audited says how many samples it kept.
4) **What to do** (Section VI). Checks for benchmark authors and readers, each with what it buys where we measured it, and one fix that keeps the waits out of the latency in any tool. Timed between the kernel's timestamps as each message leaves and returns, the latency held still across load and priority. A tool can read those timestamps on its own sockets in four steps. Better clock synchronization cures neither problem.

II. How a Message Is Timed

A producer process publishes messages to a broker at a paced rate, and a consumer process reads them back. Each message crosses two legs (Fig. 1a). Leg 1 carries it to the broker, which writes it to its log and returns an acknowledgment along leg 1, and leg 2 carries it on to the consumer. Three timestamps bracket its path: t_{pub}, read immediately before the publish call; t_{ack}, read when the producer learns that the broker has accepted the message; and t_{recv}, read when the consumer holds it. We call any difference of two timestamps a *span*. Three spans follow:

$$\begin{aligned} D &\equiv t_{\mathrm{recv}} - t_{\mathrm{pub}}, \qquad A \equiv t_{\mathrm{ack}} - t_{\mathrm{pub}}, \\ S &\equiv t_{\mathrm{recv}} - t_{\mathrm{ack}} \;=\; D - A. \end{aligned} \tag{1}$$

G. P. Ricou is with the School of Computer Science and Statistics, Trinity College Dublin, Dublin 2, Ireland. E-mail: pedrorig@tcd.ie.

R. Duvignau is with the Department of Computer Science and Engineering, Chalmers University of Technology, Gothenburg, Sweden. E-mail: duvignau@chalmers.se.

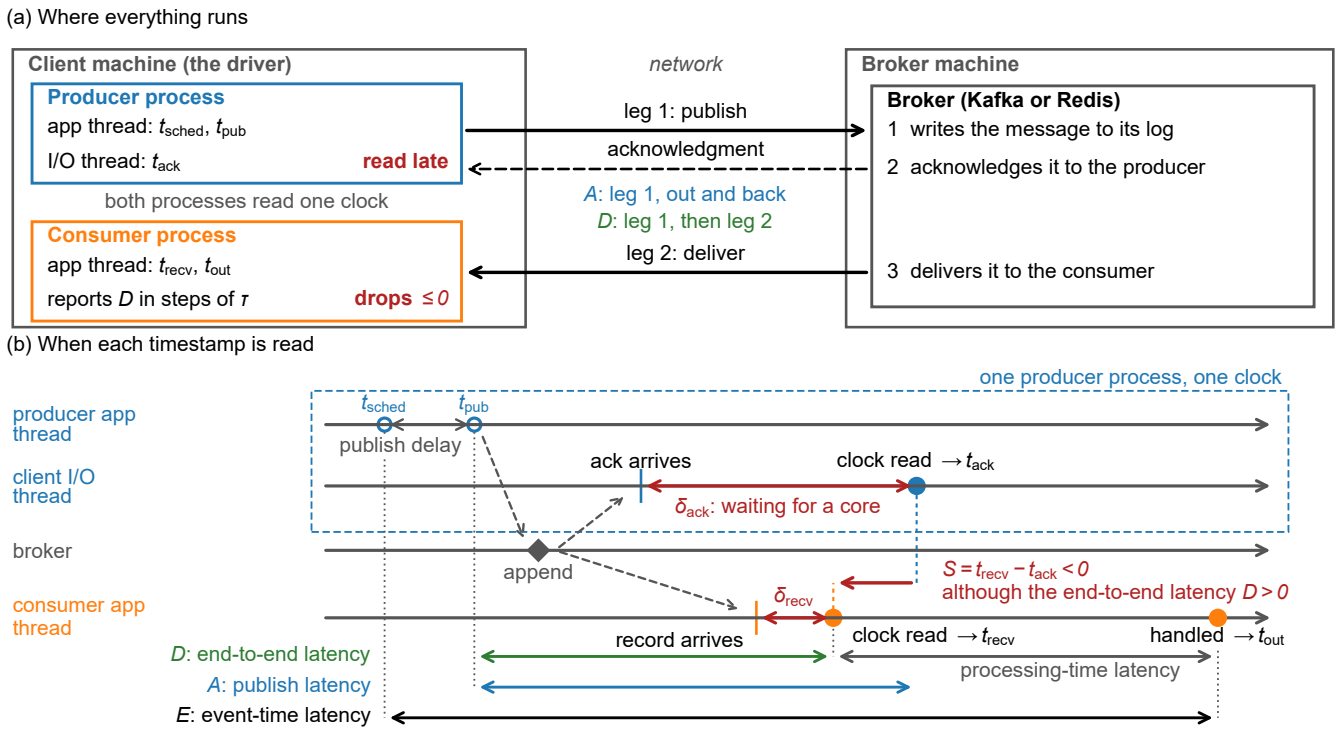


Fig. 1. **Every message crosses two legs, and each timestamp is read after its thread waits for a core.** (a) The producer and consumer processes share a client machine and its one clock, and the broker runs on a second machine. Leg 1 carries a message to the broker, which writes it to its log and returns an acknowledgment; leg 2 carries it on to the consumer. A covers leg 1 out and back, and D leg 1 and then leg 2. (b) The producer reads t_{pub} at the publish call, its client's I/O thread reads t_{ack} only after waiting δ_{ack} for a core, and the consumer reads t_{recv} after its own wait δ_{recv}. Here A runs past D, so $S = D - A$ is below zero although nothing arrived early. The red notes mark this late reading, and a tool whose clock advances in steps of τ and that drops every value at or below zero (Section V). t_{sched} and t_{out} mark when the message was due and when the consumer had handled it, and the *event-time latency* E spans them: D, with the *publish delay* before it and the consumer's *processing-time latency* after it [3]. The path drawn is Kafka's. Redis reads the acknowledgment on the application thread, which also waits for a core.

Broker benchmarks call D the *end-to-end latency*, the message's time from the publish call to the consumer, and A the *publish latency*, the producer's wait to hear that the broker has it [4]. S compares the acknowledgment's return along leg 1 with the message's trip along leg 2. A benchmark that starts its clock at the acknowledgment, as our own first one did, reports S as the message's latency.

Unless the text says otherwise, the measurements come from four Oracle Cloud virtual machines on a private network. Three run the brokers, Kafka and Redis Streams. The fourth runs the producer and the consumer as separate CPython processes that read one clock, so nothing here depends on synchronizing clocks. Its kernel is Linux 6.8, whose scheduler is EEVDF (Earliest Eligible Virtual Deadline First) [5]. The producer replays StatsBomb's open football event data [6] at a chosen publish rate, and nothing below depends on its content beyond message size and rate. Each message is a JSON record of an event's identifiers and timestamps, 253–321 bytes, which one experiment pads by up to 256 KB. A producer publishes 0.60–1.03 messages per second in nine runs out of ten, and up to 12 producers run at once. *Utilization* ρ is the fraction of processor time busy on the host over a run, and a *background load*, processor work run beside the clients, sets it. A *condition* is one point of the experimental design: a broker, a publish rate, a message size, a background load and how often the consumer confirms what it has read. A *run* executes a condition once, a *replicate* is a further run of it, and a *campaign* is a set of runs designed together. Where an experiment changes one variable and holds the rest, each value it gives that variable is a *configuration*. A later campaign ran on three pairs of Microsoft Azure machines, two x86 and one Neoverse N2. It was *registered*: its predictions were committed in public before the runs that test them [7]. Supplement S1.4 lists which runs each number rests on.

III. The Distribution of S

In the ideal case S is close to zero, because its two readings follow one write and are taken on one machine and one clock. Measured, it is spread over milliseconds, and part of it falls below zero (Fig. 2). Over the 708,505 messages of the 5,863 cloud runs that produced matched records, 62,264 values of S, 8.79%, fall below zero (excluding 30,225 messages delayed by a testbed fault; Supplement S1.5). We call this share the *negative-span rate*. It pools runs at every load we ran, each weighted by its messages. Per run it goes from zero, in 1,314 runs, to 55%, with a median of 3.6%, and Section IV shows what moves it. Both brokers show it, Kafka on 8.87% of its 359,766 messages and Redis on 8.71% of its 348,739. The registered campaign found it again in Kafka's own Java client,

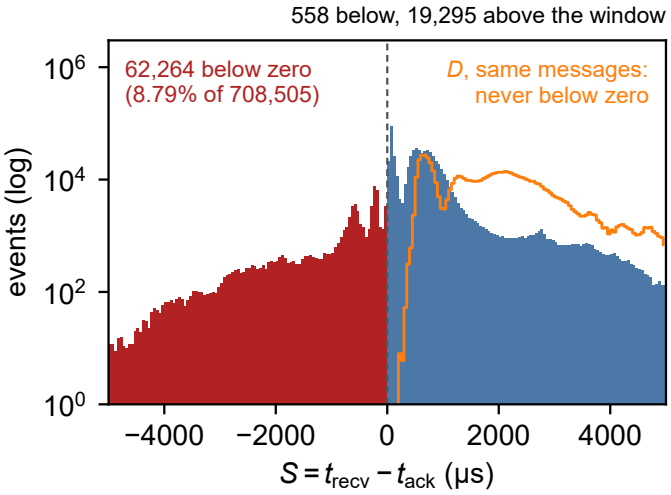


Fig. 2. S **spreads over milliseconds, and part of it falls below zero.** $S = t_{\text{recv}} - t_{\text{ack}}$ for the 708,505 messages of our cloud runs on both brokers, read on one clock at nanosecond resolution and drawn in 50 μs bins over ± 5 ms. The 62,264 values below zero are red; all but 558 of them lie inside the window, and 19,295 values lie above it. The orange line is D, timed from the publish call on the same messages, which is never below zero.

TABLE I
ONLY S FALLS BELOW ZERO, AND UNDER BOTH BROKERS. VALUES BELOW ZERO BY SPAN AND BROKER, IN ALL 5,863 CLOUD RUNS WHOSE RECORDS COULD BE MATCHED, ON ONE CLOCK. THE SMALLEST RUN MINIMUM OF THE PUBLISH LATENCY A IS 112 μS, CLEAR OF ZERO.

Span	Origin timestamp	Kafka	Redis	All
Messages		359,766	348,739	708,505
$S = t_{\text{recv}} - t_{\text{ack}}$	acknowledgment	31,899	30,365	62,264
share below zero		8.87%	8.71%	8.79%
$D = t_{\text{recv}} - t_{\text{pub}}$	publish (chain)	0	0	0
$t_{\text{out}} - t_{\text{pub}}$	publish (chain)	0	0	0
$A = t_{\text{ack}} - t_{\text{pub}}$	publish (chain)	0	0	0
$E = t_{\text{out}} - t_{\text{sched}}$	event time	0	0	0

at a lower rate, and on Neoverse N2 processors as on x86 (Supplement S6.2).

Nothing arrived early. On the same messages and the same clock, neither D nor A falls below zero once (Table I). One reading was taken late. A user of a widely run broker benchmark whose average publish latency exceeded its average end-to-end latency, with producer and consumer on one node, was told that the clocks were skewed between nodes [8]. On one clock there is no skew to blame.

IV. THE MECHANISM AND ITS LAWS

A thread must wait for a core before it can read the clock. That wait is its *scheduling delay*, the time a thread that is ready to run spends queued before a core runs it, which real-time Linux measures as scheduling latency [9]. We show by *manipulation*, moving one variable with the rest held fixed, that this wait puts S below zero, and state the laws that govern it: when S falls below zero, how often, and on what timescale.

A. When S falls below zero

D and A are causal chains. Each starts at the publish call, which precedes the broker's write, which precedes both the acknowledgment and the delivery. S is not a chain: its two readings are two branches of one cause, the write, and neither has to come first. Since Equation 1 holds for every message, whatever the signs,

$$S < 0 \quad \text{if and only if} \quad A > D. \tag{2}$$

If neither thread waited, S would equal the *margin* m, the time the message still needs to reach the consumer once the acknowledgment has reached the producer. Each reading is late by its thread's wait for a core (Fig. 1b), so S falls below zero when the acknowledging thread's wait, δ_{ack}, outlasts the margin and the consumer's own wait, δ_{recv}, together.

B. How often, and what moves the rate

Each thread that reads the clock is either running, in which case its reading is late by a narrow jitter, or waiting for a core. From an idle host to the heaviest load before its processors are fully busy, the negative-span rate grows 61× but the jitter's width only 5×, so load mostly changes how often the thread is waiting. Write p for that probability, the thread's *waiting probability*, and G for the chance that the rest of its wait, the *residual wait*, outlasts a given time. To leading order, when the consumer's own wait is short,

$$\Pr\big[\, S < 0 \mid m \,\big] \;\approx\; p\,G(m), \tag{3}$$

the waiting probability times the chance that a residual wait outlasts the margin. Load, its placement and priority move p, and the path's length moves the margin.

Priority. Real-time priority for the threads that read the clock should cut their waiting probability while utilization stays fixed. Every manipulation ran both brokers, 25 runs a configuration on each (Table II). Across eight matched pairs from 60 to 95% utilization, real-time priority cut the rate 7- to 80-fold on Kafka and 6.0- to 7.8-fold on Redis. Every pair had disjoint intervals, and achieved utilization matched to within 0.002 (Supplement S3.1). That refutes any account in which utilization alone sets the rate, including the queueing form we had ourselves adopted. In the registered campaign, on other machines, real-time priority removed the values below zero on Kafka in every run, and load raised the rate in all five Kafka campaigns that tested it (Supplement S6.2).

Placement. With the background load concentrated on six of the eight cores or spread over all of them, two configurations reached the same utilization, $\rho = 0.7531$, and on both brokers they gave different rates (Supplement S3.2). On Kafka the spread load gave the higher rate, 2.07-fold [bootstrap over runs, 95%: 1.36–3.60], and 2.02-fold [bootstrap over runs, 95%: 1.33–3.49] in a full replication. Spreading the load leaves no core free, so a woken thread waits more often. On Redis it was the concentrated load that gave more values below zero, so where the load is placed matters on both brokers, in opposite directions; we have not tested why.

Path length. A wait of a given size overlaps a short path almost entirely and a long one hardly at all. Padding the

TABLE II
EVERY MANIPULATION MOVES THE RATE ON BOTH BROKERS, AND PLACEMENT MOVES IT IN OPPOSITE DIRECTIONS. EACH FACTOR IS ONE CONFIGURATION'S NEGATIVE-SPAN RATE OVER ANOTHER'S, FROM 25 RUNS AND 2,408–2,985 MESSAGES A CONFIGURATION, WITH A 95% INTERVAL THAT RESAMPLES WHOLE RUNS. UTILIZATION IS MATCHED WITHIN EACH COMPARISON EXCEPT LOAD'S. PRIORITY GIVES THE RANGE OVER THE PAIRS, EACH WITH DISJOINT INTERVALS ON ITS TWO RATES (SUPPLEMENT S3.1). PADDING GIVES THE HIGHEST RATE OVER THE LOWEST ALONG THE PADDING.

Manipulation, and the ratio	Kafka	Redis
Real-time priority, eight pairs normal / real-time	7–80	6.0–7.8
Placement at $k = 6$ spread / concentrated; repeat	2.07 [1.36–3.60] 2.02 [1.33–3.49]	0.65 [0.55–0.78] 0.62 [0.51–0.76]
Padding the payload highest / lowest rate; repeat	3.8 [3.1–4.8] 4.0 [3.3–5.0]	7.9 [6.2–10.4] 5.7 [4.4–7.7]
Load, idle to 88% loaded / idle	61 [33–215]	21 [14–36]

payload lengthens the path without touching the scheduler. On Kafka it raised the median of S 74-fold while utilization varied by 0.012, and the rate fell 3.8-fold; on Redis the rate fell 7.9-fold. That is the direction G predicts, and the opposite of what stress on the host would cause (Supplement S3.3).

An independent check. A kernel trace of the scheduler's wakeup and switch events [10] times the wait directly and shares no probe or data with the application's timestamps. It counts the threads of both brokers' runs, so it is compared with both brokers' rate. With nothing fitted, the share of traced wakeups that waited longer than 0.5 ms, our estimate of the margin in those runs, stands to that rate at ratios of 0.84, 1.01 and 1.17 in the three normal-priority configurations traced (Supplement S3.5). The registered campaign timed each acknowledgment's own wait. Of the readings below zero, 79.4–92.0% had spent most of the time between the reply's arrival and the timestamp waiting for a core. The share of acknowledgments whose wait outlasted the margin stood to the rate at medians of 1.11–1.30 (Supplement S6.2).

C. The timescale the slice sets

A thread woken while another holds the core waits out what is left of that thread's *slice*, the time the scheduler lets a thread run at a turn, up to a whole one. The kernel sets the base slice s from the number of CPUs n, with a cap:

$$s \;=\; s_0 \times \big(1 + \lfloor \log_2 \min(n, 8) \rfloor\big), \tag{4}$$

where s_0 is 0.75 ms in Linux 6.8 and 0.70 ms after a change of 2025 [11], which our distribution's kernels carry. That gives 4×0.70 ms $= 2.8$ ms on our hosts' eight CPUs. The traced waits carry that timescale. Over 551,956 wakeups, their counts have three local maxima, which survive a 2× widening of the buckets. The last sits in the 2–4 ms bucket that contains the slice, with 10.5% of all wakeups, 4.5× the bucket below it, and above it the counts collapse (Supplement S3.7). The trace counts every wait, including those that begin when a thread is preempted, which last about a whole slice. The thread that reads the acknowledgment is instead woken by the reply. The registered campaign timed that thread's own wait for every

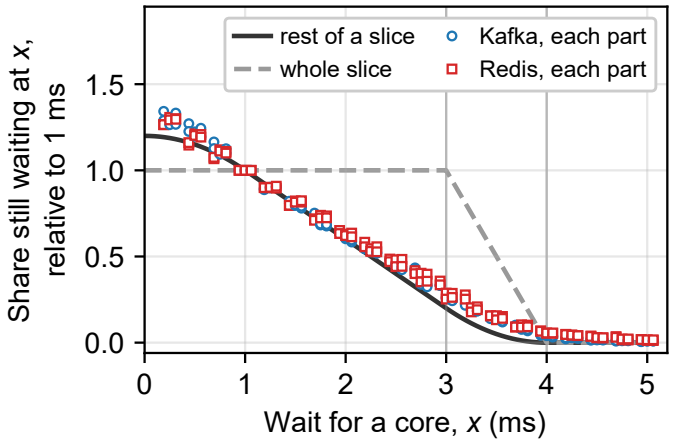


Fig. 3. **Woken by the reply, a thread waits out the rest of a slice, not a whole one.** For each of the registered campaign's twelve combinations of machine pair, broker and load, the share of acknowledgments whose thread was still waiting for a core at x, relative to the share still waiting at 1 ms, over 1,516,342 acknowledgments in 304 runs. Those runs set the slice to 3 ms and the *tick*, the timer interval at which the kernel checks whether a slice has run out, to 1 ms. The solid curve waits out the rest of a slice and then up to a tick (Supplement S6.3); the dashed curve waits out a whole slice and a tick. The vertical lines mark the slice and a tick past it. Read after the campaign's verdicts, on runs whose recording raised the rate it recorded 1.36- to 4.33-fold.

acknowledgment, and a reading made after its verdicts found it to be the rest of a slice (Fig. 3).

The CPU count moves the slice. In the registered campaign the slice read back at 2, 4 and 8 CPUs as Equation 4 gives with $s_0 = 0.70$ ms. On Kafka, at the one CPU count whose slice is at least its shortest end-to-end latency, the end-to-end latency at which the rate falls moved with the slice (Supplement S6.2).

A whole slice is 0.95–4.00 times the per-condition median end-to-end latencies we measured, 0.70–2.95 ms, and the margin is shorter than the end-to-end latency, so the slice lands on the scale of the paths now benchmarked. Read together, Equations 2, 3 and 4 say what the rate depends on. It rises with the waiting probability p, which load raises, the placement of the load moves, and real-time priority lowers. It falls as the margin outgrows the residual wait, whose scale the slice sets and the CPU count moves. On our kernel, any host with at least as many CPUs as ours has a slice of 2.8 ms, so a single wait can outlast any margin shorter than that.

V. AN INDUSTRY-WIDE AUDIT

What do the industry's tools do with a latency at or below zero, and with one shorter than their clock's step? We read 27 tools at source, ran 32 under registered predictions, measured one benchmark's filter in depth and audited the public reports made with it. The four agree. Such a value is deleted, replaced or never measured, and almost never counted (Table III).

A. Tools read at source

We read 27 measurement tools at source, across seven languages and 20 vendors, and classified them with one program whose verdicts, each with its source line, are in the supplement's tool registry (Supplement S5). Of these, 24 time

TABLE III

WHAT THE TOOLS WE READ AND RAN DO WITH A LATENCY AT OR BELOW ZERO. THE TOOLS ARE GROUPED BY WHERE THEY TAKE THEIR TWO TIMESTAMPS. ONLY THE FIRST GROUP MEASURES A ONE-WAY LATENCY (SECTION V-B). WITHIN EACH GROUP, ROWS RUN IN THE ORDER DROPPED, REFUSED, REPLACED, AND KEPT OR NEVER MET. STEP: THE SMALLEST CHANGE IN THE LATENCY A TOOL REPORTS. READ: CLASSIFIED FROM ITS SOURCE (SECTION V-A). RUN: RUN IN THE REGISTERED CAMPAIGN (SECTION V-B). KAFKA'S TWO BUNDLED TOOLS COUNT AS ONE TOOL READ BUT TWO TOOLS RUN. SOURCE LINES, VERSIONS AND DATES: SUPPLEMENT S5.

Tool	Step	What happens to a value at or below zero	Read, run
Timestamps taken in two processes, as a one-way latency needs			
OpenMessaging Benchmark	1 ms	Dropped by a filter that keeps values above zero, uncounted; at that step the filter also drops every message faster than a millisecond	read, run
Apache Pulsar's performance client	1 ms	Dropped when below zero, uncounted; a zero is kept	read, run
librdkafka's `rdkafka_performance`	10 μs	Dropped unless above zero, with a printed line, uncounted; none reached zero in our runs	read, run
RabbitMQ Stream PerfTest, as producer and consumer	1 ms	Dropped when below zero, uncounted, since June 2026; a zero is kept	read, run
Yahoo Streaming Benchmark	1 ms	Dropped by the notebook that analyzes its output file, uncounted; kept in that file	read, run
Apache ActiveMQ Artemis's `perf`	1 μs	Refused by its histogram library; the session is rolled back and the run still reads success; uncounted	read, run
Google's Pub/Sub load test framework	1 ms	Replaced by the bin of the fastest messages in its Java client, and the controller reading those bins throws, blaming negative values, once half the values lie there; its Go client panics, its Python client raises, its Node client loses it; uncounted	read, run
Kafka's Trogdor, end to end	1 ms	Replaced by its histogram's floor, uncounted; that histogram throws on a negative, which loses a producer's sample	read, run
Quiver	1 ms	Replaced by some 10^{19} ms when unsigned arithmetic wraps it, uncounted	read, run
`emqtt-bench`	1 ms	Replaced by zero in its average, which is printed only when some value is above zero; kept in its histogram since January 2025	read, run
PerfKit Benchmarker, messaging	1 ms	Kept, floored to whole milliseconds, also when a step of the clock makes one	read, run
Redpanda's `kgo-repeater`, RocketMQ's benchmark	1 μs, 1 ms	Kept, unflagged; RocketMQ's maximum, which starts at zero, cannot show it	read, run
Both timestamps in one process: publish to receipt, or to the acknowledgment			
Strimzi Canary	1 ms	Dropped by the log reader it ships, uncounted; binned with the fastest messages	read, run
Apache BookKeeper's benchmark, to the acknowledgment	1 ns	A zero dropped with the failed writes, uncounted; no value below zero can arise	read, run
Storage Benchmark Kit (SBK)	1 ms	Dropped from its figures but counted, a zero kept; the one message tool that counts what it cuts	read, run
NATS command-line latency test	1 μs	Refused by its histogram library for its percentiles, kept for its minimum, median and maximum; uncounted	read, run
Pravega benchmark	1 ms	An array index that throws, ending every later statistic; uncounted	read, run
Kafka's end-to-end and producer tools	1 ms	End-to-end tool: an arrival before its acknowledgment replaced by the time of the poll that follows the acknowledgment, uncounted; on a path faster than a millisecond all its percentiles read zero. Producer tool: every value kept, unflagged	read, run
RabbitMQ PerfTest	1 μs	Replaced by its absolute value, uncounted; until June 2026 a zero was dropped too	read, run
Xinfra Monitor	1 ms	Replaced by zero in its percentiles, kept in its average; uncounted	read, run
Sarama's producer tool, requests; `mqtt-benchmark`	1 ms, 1 ns	Never met: a monotonic clock in one process; Sarama truncates every request under a millisecond to zero	read, run
Both timestamps in one process: request to reply, a round trip			
`wrk2`	10 μs	Printed as impossible, then refused by its histogram library, or the run aborts; uncounted	read, run
`hey`, `k6`, `memtier`, Valkey benchmark, Vegeta, YCSB, `nats bench`	1 ns–0.1 ms	Cannot fall below zero in one process; none did when the clock was moved back	run
A curiosity: intervals inside one host, which time no message			
Rezolus, scheduler wakeups	1 ns	Dropped and counted, since August 2026	read
`fio`, input and output completion	1 ns	Replaced by zero, uncounted	read
`btt`, block-layer stages	1 ns	Replaced by 1 ns, also when the two timestamps are equal; uncounted	read

messages. Of those, 18 cut a value at or below zero out of a figure they report, without counting it, in three ways: a filter drops it, the tool's own library refuses it, or another value replaces it. Replacement hides best, because nothing looks missing. RabbitMQ PerfTest takes the absolute value [12], and `emqtt-bench` keeps every sample in its histogram but counts such a value as zero in the average it prints [13]. Until June 2026 PerfTest also kept only values above zero, which in its millisecond mode deleted every trip shorter than a millisecond. The filter in librdkafka's `rdkafka_performance` prints a line for each value it drops but keeps no count. Users have reported such values: in the Yahoo Streaming Benchmark's tracker in 2018, still open, and in RabbitMQ Stream PerfTest's in May 2026, which answered with a filter that drops them. Apache Kafka's own end-to-end tool hides a value of S below zero without computing one. It polls for each message only once the acknowledgment has returned, so a message that reached the consumer before its acknowledgment is timed to that later moment, which replaces its arrival. Beside an observer that noted when each message became readable, 11–41% of messages were readable before the tool had their acknowledgment with every core 88% busy, and at

most 0.2% with the tool at real-time priority (Supplement S5). It also rounds each latency down to whole milliseconds, so on a path faster than a millisecond every percentile it reports is zero [14]. SBK cuts such a value out of its figures, but counts it and prints the count. Of the five message tools that cut nothing, two time only to the acknowledgment, on a monotonic clock, and never meet such a value, and three keep it, PerfKit Benchmarker's messaging benchmark floored to whole milliseconds (Supplement S5). Of the tools that time no message, `fio` and `btt` substitute zero and one nanosecond, and only Rezolus counts what it discards, in its scheduler sampler since August 2026 [15].

B. Tools run

Reading a tool shows what it does with a value below zero, not whether it ever meets one. The registered campaign therefore ran 32 benchmarking tools, four rounds each, with predictions fixed from their source before any run. PerfKit Benchmarker and the Pub/Sub load test ran against managed services, the others against a broker on a second machine. Twenty-one of them ran in October, in blocks of their own: five the plan had once left out, and sixteen read at source that month (Supplement S5). A delay added to the path in steps cannot reach a managed service. It moved the figure of every other tool but two 0.56–1.48 ms per millisecond added and per crossing of the delay, so those tools measure the path. The Yahoo Streaming Benchmark's did not, as predicted, though for another reason: each write of its results waits for four replies that cross the delay. RocketMQ's benchmark's did not either, against its prediction. Some of its runs waited about 20 s in the broker until their consumer was given its queue, and in the rest its consumer, which asks the broker for each batch, met the delay about twice (Supplement S5). Moving back the clock of the process that subtracts, at two offsets longer than the interval the tool times, separated the tools that read one clock from those that read two. The figures of 19 tools stayed within a quarter of the offset in 151 of their 152 runs, because each reads both timestamps from one clock; in the other, SBK's minimum, which it prints in whole milliseconds, moved by one. All but PerfKit Benchmarker, whose two processes share one machine, take both timestamps in one process, timing a round trip or a message they publish and receive themselves, so they cannot measure the one-way latency of the delivery a broker exists to provide. The Pravega benchmark reads one clock too, but it counts only what arrives in a fixed minute, so its count fell short whether or not its clock was moved, and most of its runs read as if the moved clock had cut what it kept (Supplement S5). A constant offset cannot make a value below zero in any of these tools, but a step of the clock can. Stepping the wall clock back under six tools that time on one clock showed what each does with such a value, in all four rounds and as their sources predicted. PerfKit Benchmarker keeps it, Strimzi Canary logs it, SBK counts it, Stream PerfTest, run as one process, drops it uncounted, and Xinfra Monitor's metrics library pins it to the floor of its percentiles, saying so only at debug level. The Pravega benchmark stops counting at the first one, so it prints no total, and its code then exits as if the run had succeeded (Supplement S5). The other ten timestamp a message in one process and subtract in another, Redpanda's `kgo-repeater` on another machine. In 73 of their 80 runs the moved clock shifted their figures or cut what they kept, and the rest were undecided. Of these, librdkafka's `rdkafka_performance`, `kgo-repeater` and RocketMQ's benchmark add the clocks' offset to the path's latency. Pulsar's client, the OpenMessaging Benchmark, RabbitMQ Stream PerfTest and the Yahoo Streaming Benchmark drop what comes out below zero, uncounted, and Kafka's Trogdor replaces it with zero. Under the larger offset Pulsar's client kept 3 to 7 of $3{,}000$ latencies, and Stream PerfTest 3 to 4. Quiver keeps both timestamps of every message in a file, but its summary subtracts in unsigned 64-bit integers, so a value below zero becomes about 1.8×10^{19} ms, some 585 million years, as its maximum did in every run under the offsets (Supplement S5). Apache Artemis's perf client also subtracts in a second process, but its consumer refuses a value below zero and has the message delivered again until it comes out above zero. Its figures therefore hid the offset, while it received $5{,}992$ to $8{,}972$ messages from the $3{,}000$ its producer publishes in a run, and reported success. With the clock stepped, its producer stopped after 2 to 10 messages and again reported success (Supplement S5). Pub/Sub's load test reads two clocks too, but bins every value at or below a millisecond with its fastest messages. Under both offsets at least half its subscriber's values fell there, so its controller threw an exception blaming negative values and printed no latency for the subscriber (Supplement S5). `emqtt-bench`'s histogram keeps values below zero, but its console printed no latency under either offset (Supplement S5). Waiting for a core shows in these tools as time added rather than as a value below zero. With every core 88% busy, the 99th percentile of each of the 25 tools whose figures can show such a wait rose by 0.9–7.0 ms, and real-time priority took nearly all of that rise back in 17 of them. Medians moved by at most 0.2 ms, except in four tools that print whole milliseconds, where they rose by up to four milliseconds and real-time priority put them back where they were idle (Supplement S5).

C. A positivity filter on a millisecond clock

The positivity filter is one of the dispositions above, and the one that deletes more than the values below zero. In statistical terms it truncates the distribution at zero: what it removes is not counted, as it would be if censored. We measured it in depth in a broker benchmark that many public comparisons run, the OpenMessaging Benchmark [4]. Its filter was added in 2018 because its end-to-end latency "is calculated based on time difference of `System.currentTimeMillis()` on two different machines" and "can be negative" [16], and its histogram rejects values below zero outright. For clocks that disagree, the filter is a sound local fix.

It has two side effects. Its clock counts whole milliseconds (whole seconds in its RabbitMQ driver; Supplement S5), so its *timestamp resolution* τ, the smallest step a timestamp can take, is a millisecond. A message that arrives before the millisecond digit changes therefore computes to zero and is

deleted (Fig. 4a). And nothing counts what the filter deletes. A sample survives only if a millisecond boundary falls between its two timestamps. With publish instants placed uniformly within the millisecond, that happens with probability T_{true}/τ on a path shorter than τ, where T_{true}, the *true end-to-end latency*, is the one a perfect clock read without delay would give. So the share of samples kept, the *retention*, is

$$\text{retention} \;=\; \min\!\left(1,\; \frac{T_{\mathrm{true}}}{\tau}\right). \tag{5}$$

Every sample kept on such a path reads exactly one step, so two brokers whose messages all arrive within a millisecond print the same median and differ only in their retention, which is never printed.

We added counters that record, by sign, every sample the filter discards, and changed nothing else. Every run was in the benchmark's embedded mode, which runs it whole in one Java virtual machine. Of the 75 settings whose own summary we captured, 71 printed a median of exactly 1.0 or 2.0 ms, computed from 0.36–100% of the samples taken (Fig. 4b). Two replicates of one setting kept 431 and 120,434 samples, 279-fold apart, and printed the same median, with nothing in the output to tell them apart. The filter can also inflate a median. At 64 KB, the two settings kept only 34.4–35.9% of their samples, and every sample they deleted computed to zero, so most of their messages arrived within a millisecond. Yet they printed medians of 235 and 519 ms, those of the slow remainder.

Equation 5 holds where it can be tested. It predicts one retention for every configuration whose publish interval bears no ratio of small whole numbers to τ. The seven such configurations all keep 47.2–51.5%. That puts T_{true} at 0.49 ms, read as a mean over messages with any path longer than τ counted as τ. Where the ratio is one of small whole numbers, the periodic schedule phase-locks with the clock: the publish instants visit only a few positions within the millisecond, and retention takes only a few values. A registered manipulation of message size moved one configuration onto one of those values and off it (Supplement S4.3). The arithmetic is in Hewlett-Packard's 1970 counter note [17], but the deletion is not: the counter keeps its zero readings and estimates the interval from the share that are not zero.

Signed counters show the filter hiding two different failures. On Kafka, the 10,913,263 samples it discarded over 214 runs hold 0 values below zero, so every discard is a loss of resolution. On Redis, the benchmark subtracts a millisecond timestamp the broker assigns from the consumer's millisecond clock on another host. All 41,403 values below zero over its 9 runs are exactly $-\tau$, the only value below zero that two floored clocks less than a millisecond apart can produce (Supplement S4.4). An unsigned counter cannot tell the two failures apart.

D. Public reports

We registered an audit of the benchmark's public reports before reading any (Supplement S5). Its signature is a printed end-to-end median of exactly 1 ms, the smallest value the benchmark can print and the value every kept sample takes on a path faster than a millisecond. Of 43 public reports that print the benchmark's end-to-end percentiles, across 1,100 configurations, eight of the 868 medians they print carry it, in two reports. None of the reports states how many samples it kept. The signature is rare partly because 9 of the reports come from copies of the benchmark whose maintainers changed how the span is timed, and two of them print medians below a millisecond, which the original benchmark's clock cannot produce. Those copies still take the publish timestamp in whole milliseconds for Kafka and Pulsar, and keep the positivity filter (Supplement S5).

TABLE IV
MOST STEPS COST LITTLE, AND MOST WERE MEASURED. WHAT A BENCHMARK AUTHOR OR READER SHOULD DO, AND WHAT EACH STEP BUYS ON OUR RUNS.

Do this	What it buys
Time each message between the kernel's timestamps as it leaves and as it arrives, read on the tool's own sockets.	Holds no client thread's wait. Its median moved at most 7.8% across load and priority, while load at normal priority raised the tail of D 4.2–4.7× (Supplement S3.10).
Sign-check every run under the load it is quoted at, and report the rejection rate.	Catches runs in which A exceeds D, and rejected 51.9% of our cloud runs. Misses a latency that load inflates but leaves positive.
Time from the publish call, or record the publish latency and add it back.	Recovers D to a median error of 3.4% where the sign check passes and 12.2% where it fails. S alone understates D 4.2× at the median (Fig. 6).
Give the threads that read the clock real-time priority, or have them spin on a dedicated core (*busy-polling*).	7–80× fewer values of S below zero on Kafka and 6.0–7.8× on Redis, in each of eight matched pairs (Table II). Busy-polling: not measured.
Record at native resolution, and print the retention and the discards counted by sign beside every latency.	Shows what a filter deleted, and whether the resolution or a clock failed (Section V-C).
Read your own percentiles: on a fast path, a median of exactly one or two clock steps was set by the clock.	Flags 71 of the 75 settings we captured (Section V-C).
Randomize the publish instants (Poisson sampling), and record the achieved rate.	Keeps the schedule from phase-locking with the clock. Not measured.

VI. WHAT TO DO

The late reading, a clock read delayed while its thread waits for a core, and the deletion of values at or below zero are cheap to detect, and most of what they cost is cheap to repair (Table IV). One fix ends the late reading in any tool that times messages: take both of a message's timestamps from the kernel, not from the tool's threads (Fig. 5). It comes first, and the rest find or shrink the late reading in a tool that still reads the clock in its threads.

Let the kernel timestamp each message. A tool reads each of a message's timestamps in one of its threads, and that thread may first wait for a core. The kernel timestamps each packet as it leaves a machine and as it enters one, the transmit and receive timestamps, without waiting for any program's thread. A latency timed from a message's transmit timestamp to its receive timestamp, the *kernel-timed latency* W, therefore holds

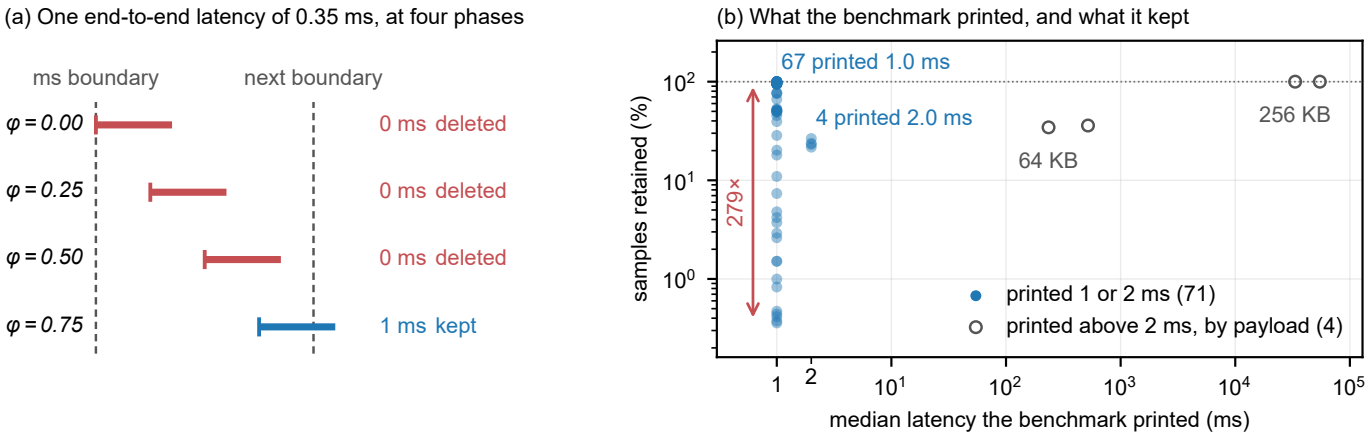


Fig. 4. **Why a sample is deleted, and what the deletion did.** (a) One end-to-end latency of 0.35 ms, starting at four phases within one millisecond. Three finish inside it, compute to 0 ms and are deleted. The fourth crosses a millisecond boundary, where the clock's digit changes, and is kept, as exactly 1 ms. (b) In the benchmark's embedded mode, each of the 75 settings whose summary we captured, at the median the benchmark printed and its retention. Of them, 71 print 1.0 or 2.0 ms while their retention spans 279×. Of the settings that print more, only those at 256 KB keep every sample.

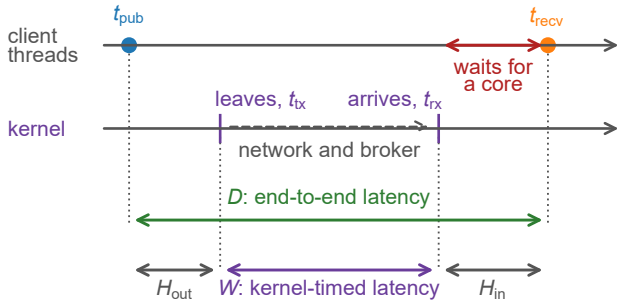


Fig. 5. **Timed between the kernel's timestamps, a message's latency holds no thread's wait.** The client's threads read the clock at the publish call, t_{pub}, and at receipt, t_{recv}, and the consumer's thread first waits for a core. The kernel timestamps the message's packet as it leaves, t_{tx}, and as it arrives, t_{rx}, without waiting for either thread. The latency between them, W, holds the network and the broker. Any thread's wait falls in H_{out} or H_{in}, and $D = H_{\mathrm{out}} + W + H_{\mathrm{in}}$ for every message.

the network and the broker and none of the clients' waits. The waits are not lost. They fall before and after W: H_{out} runs from the publish call to the transmit timestamp, and H_{in} runs from the receive timestamp to the consumer's reading, so that $D = H_{\mathrm{out}} + W + H_{\mathrm{in}}$ for every message (Fig. 5). A tool can therefore report what the network and the broker did, W, beside what the application saw, D.

All five predictions we fixed before any run held. Over 48 runs on both brokers, timed from packet captures, the median of W moved by at most 7.9% and its 99th percentile by at most 7.2% between no background load and 88%, and between normal and real-time priority. In the same runs, at normal priority, the load raised the 99th percentile of D 4.3–4.6-fold (Supplement S3.9). No kernel-timed latency fell to zero or below, so a positivity filter has nothing to delete.

Read those timestamps on the tool's own sockets. A tool need not capture packets. On Linux it reads the kernel's timestamps in four steps, through the `SO_TIMESTAMPING` *socket option*, a setting a program makes on one of its sockets.

1) On each connection to the broker, before its first write, turn on the kernel's software timestamps for transmit and receive, with an identifier for each write.
2) The kernel leaves each write's transmit timestamp on the socket's *error queue* once the write has gone out. The identifier is the byte offset of the write's last byte, which names the messages the write carried. Collect at every write and read, because while the queue holds a timestamp, the socket reports an error, and a client waiting on it spins.
3) With each read, take the receive timestamp the kernel returns beside the data, that of the last packet the read consumed.
4) Time each message from the transmit timestamp of the write that carried it to the receive timestamp of the read that delivered it, and report that latency beside D, counting any message left without a timestamp.

Our Python module takes these steps beneath both brokers' client libraries, which it leaves unchanged, and finds each message in the bytes they write and read (`scripts/so_timestamps.py`). In 48 more runs, the kernel-timed latency read this way came within 3.0 μs of the captures' at the median. Its median moved by at most 7.8% across load and priority, and the tail of D again rose 4.2–4.7-fold under load at normal priority (Supplement S3.10).

What a tool must handle. A write or a read that carries several messages gives them the timestamp of its end. A read that drains several packets therefore times the earlier ones late, and a consumer that waits while packets queue adds part of its wait back to them. At our publish rate the receive timestamps matched the captures' for 98.4% or more of each run's messages. Where a driver gives no software transmit timestamp, as on our virtual network, ask instead for the kernel's timestamp as a packet enters the *device queue*, where it waits for the driver. These came 2.3–3.2 μs before the capture's, in each run's median. The kernel timestamps an arriving packet a median of 6–20 μs after the network card does (Supplement S3.9), and a card that timestamps in hardware would remove that delay, on a clock of its own that

the tool must tie to the system's. With the producer and the consumer on two machines, W spans two clocks and, like any one-way latency, includes their offset. Java's standard socket classes expose neither the option nor the error queue, so a Java tool needs a native call. A tool that cannot yet take the kernel's timestamps needs what follows.

Sign-check every run. The *sign check* rejects a run when more than one percent of its messages carry a value below zero in any latency component it records (S, the publish delay or the processing-time latency), or when the median of any of them is below zero. It rejected 459 of the 884 cloud runs behind our results (51.9%; Supplement S2). It sees only the worst cases. The publish latency exceeds half the end-to-end latency on 77.6% of messages, but the check sees only the 8.8% on which it exceeds the whole, and a latency that load inflates but leaves positive passes by construction. The check therefore needs the repair that follows.

Time from the publish call, or add the publish latency back. The publish latency does not grow because a message travels further, so on one path, at a fixed publish latency, the relative error of S as a latency, A/D, falls as $1/D$. At our median publish latency, $A = 650\,\mu$s, the error is 65% at an end-to-end latency of 1 ms and 6% at 10 ms, and below 0.65 ms the median publish latency exceeds the end-to-end latency itself (Fig. 6). Across the band of A in the figure, the error at 10 ms runs from 5 to 19%. From one condition to the next, though, the publish latency grows with the end-to-end latency, so our conditions sit above the curve. Grouping each condition's runs by the sign check's verdict gives 70 groups from 51 conditions, over which S understates the end-to-end latency 4.2× at the median, with an interquartile range of 4.0–7.5×. Because the two brokers' clients read the acknowledgment differently, S also distorts the gap between them, a property of the clients that a reader would credit to the brokers. Over 24 matched workloads it inflates the gap 1.7× at the median, with an interquartile range of 1.3–2.0×, and in 21% of them it narrows the gap instead.

The repair is Equation 1. $D = S + A$ holds for every message, and A is read on the producer's side of the same clock, so a benchmark that records its publish latency recovers, message by message, the end-to-end latency it failed to measure. A benchmark that keeps only medians can add only those, and the median of S plus the median of A need not be the median of D. That recovery's median error is 3.4% on the runs the sign check passes, in 42 conditions, and 12.2% on the runs it rejects, in 28, with upper quartiles of 20% and 19%. Timing from the publish call removes the values below zero, but not the consumer's own wait for a core, which lengthens every latency and which no check sees. Measured three ways in 51 later runs on the registered campaign's machines, that wait added 0.41–0.94 ms to the mean end-to-end latency at 75 and 88% background load, while the median message waited 18–46 μs (Supplement S3.8). The kernel-timed latency keeps that wait out.

Give the threads that read the clock a core. Real-time priority cut the negative-span rate 7–80× at unchanged background load on Kafka, and 6.0–7.8× on Redis (Table II). A busy-polling thread spins on a core of its own instead of sleeping, and never waits to be woken. Busy-polling a dedicated core should buy the same for the price of that core, though we did not measure it. Since Linux 6.12 a thread can also ask for a shorter slice of its own, so that it runs sooner when woken, without real-time priority [18]; we did not measure that either. At real-time priority the rate settles at 0.0017–0.0144 on Kafka and 0.0176–0.0255 on Redis, not at zero, so the check stays, and a real-time or polling thread changes the system it measures.

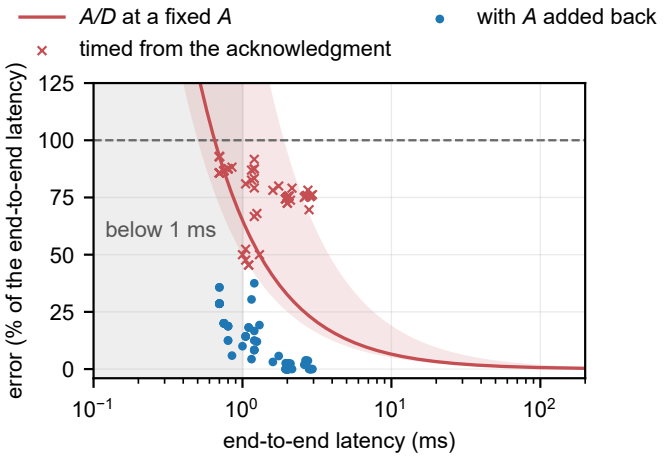


Fig. 6. **Adding the publish latency back repairs the latency.** The error of S as a latency on each of our 70 groups of runs, one per condition and sign-check verdict (crosses), and once the publish latency is added back (dots), against the group's end-to-end latency. The curve is A/D for one path at the median publish latency, and its band covers the tenth to ninetieth percentiles of A across groups, 500–1900 μs. The crosses sit above it because the conditions with a longer end-to-end latency also have a longer publish latency. On the dashed line A equals D. The shaded decade holds the sub-millisecond medians now reported for brokers.

Count what the filter deletes. The deletion leaves nothing in a printed percentile, so the remedy is to make the tool print it: record at the clock's native resolution, and print beside every latency the retention and the discards, counted by sign. A discard at zero means the clock's step was too coarse for the path, and one below zero means two clocks disagreed (Section V-C). A schedule paced at a simple fraction of the clock's step can phase-lock with it, and randomizing the publish instants breaks the lock.

A better-synchronized clock repairs neither the late reading nor the deletion: the values of S below zero arise on one clock, and the deletion from the clock's step, not its offset. A finer clock confines the deletion to faster paths but leaves the filter and every value of S below zero in place. The remedies are therefore ordered: record at native resolution, count what is discarded, randomize the publish instants, and only then does synchronization quality matter.

VII. Related Work

Scheduling delay. That a ready thread can wait for a core is well known [10]. On servers, a thread kept waiting behind another's time slice adds milliseconds to the tail latency, and real-time priority removes that wait [19]; co-located work lengthens it [20], and real-time Linux measures the noise it adds [21]. Scheduler studies find the wait roughly

proportional to the time slice [22], and the limits of the timers that benchmarks read are known too [23]. Load testers of microsecond-scale services correct for the *client bias* their own queueing and scheduling add to the latencies they report [24], [25], Lancet by busy-polling. Chandrasekar and Kramberger model that bias with a single-parameter queueing law [26] that our placement contrast refutes. Villain et al. found that an outgoing socket timestamp "does not respect causality" under every stress pattern, including none, and traced host latency to interrupts, scheduling and context switching [27]. None gives the consequence for a difference read by two processes: that it falls below zero at a rate set by the waiting probability and the margin, on the timescale of the slice.

Clocks. Elsewhere, values below zero are read as clocks that disagree. Sharma et al. count negative timing spans in distributed inference systems and read them as clock skew, holding that "queueing alone cannot produce negative timing spans" [28]. For the backlog they model we agree, but a timestamping thread waits in a second queue, the kernel's run queue. Van Dongen and Van den Poel, who saw latencies below zero on up to a tenth of their events when timestamps came from different brokers, took both from one broker [29], the one-clock rule reached by another route. Better measurement systems synchronize clocks [30], calibrate counters across nodes [31] or timestamp in the network interface [25] or the switch [32], and the checks of Section VI apply to whatever they measure.

Resolution and discarded samples. The retention arithmetic and its cure, phase jitter, are in a 1970 counter note [17], and a clock coarser than the intervals it times can still estimate their mean from many readings, though not any one of them [33]. IP measurement advises randomizing probe times [34], [35]: probes at random instants see the time average, which a periodic schedule can miss [36]. Coordinated omission [37] is the deletion's mirror image: it fails to record samples that would have been slow, while a positivity filter deletes samples that were fast. Network measurement reports the share of traces it rejects [38] and keeps a packet timestamped in the future [39]. The literature on rigorous benchmark reporting [40], [41], [42] asks, in Hoefler and Belli's words, that removed outliers be counted. The tools we audited remove samples before any analyst sees them and count none, a case that neither that literature nor the principles for reproducible performance evaluation in the cloud [43] address. Research benchmarks of stream processing compute latency from two broker-assigned millisecond timestamps [44], so the retention arithmetic applies wherever their latencies fall below a millisecond. What we add is these checks, applied where a benchmark reports a latency.

VIII. Limitations

Attributing the values of S below zero to scheduling is an inference. One clock excludes clock skew, the manipulations exclude utilization, timer noise and stress, and the kernel trace observes the wait. Clocks that disagree across virtual CPUs [45] are excluded too. A clock read in 90 ns can only be `kvm-clock` or `tsc`, both of which the kernel uses only where the CPUs' clocks agree, and an offset large enough for the deepest value of S would have driven D below zero, while the smallest D observed caps any offset 455-fold lower. Both testbeds are virtual, so *steal time*, the time a virtual processor is ready but waits while the hypervisor runs another machine, is bounded, not excluded: real-time priority, a setting inside our virtual machine, cannot shorten it, yet it cut the rate. Our CPython timestamps also wait on the interpreter lock, but Kafka's Java client, which has none, shows the same values below zero. One rival is not excluded. A broker that delivered the message to the consumer before it acknowledged the producer would put S below zero with no wait at all. Kafka runs with `acks=all`, so both replies follow the same commit, but we did not capture the order in which they left (Supplement S3.6). Behind most readings below zero, the thread had spent most of its time waiting for a core (Section IV-B), and real-time priority, which cannot reorder the broker's replies, cut the rate in every pair, so this rival could explain only the rest.

The laws hold to leading order, and Equation 3 describes how the rate moves between conditions, not single messages. Read with the measured end-to-end latency in place of the margin, it would have the rate fall as D grows inside a condition, yet the rate rises in 24 of 56 testable conditions, because a long delivery comes with a long publish latency (Supplement S3.4). A registered prediction that priority would cut the waiting probability at least tenfold failed, scheduler counters seeing it fall 1.7- to 2.2-fold, so the factorization of Equation 3 is untested. The registered campaign also predicted the rate from recorded waits, to within a factor of 1.5 either way, and none of its three predictions was confirmed. Predictions from every thread's waits ran 2.49–13.08 times the measured rate, those from the time-weighted waits 0.104–0.165 times, and those from each acknowledgment's own wait met the rule on four of six combinations of machine pair and broker. The share of readings below zero whose thread had mostly been waiting reached the registered 90% on four of twelve combinations of pair, broker and load (Supplement S6.2). The tracer is not free: under it Kafka's real-time runs read no value below zero, where untraced they read 15 in $2{,}985$, though its normal-priority rate did not move (0.273 untraced, 0.272 traced). The registered campaign's recording raised the rate it recorded 1.36- to 4.33-fold. Each prediction is therefore held against the traced run's own rate. The base slice on the Oracle hosts is derived, not read off them; the Azure hosts read it back. In the registered campaign, the predictions of where the rate falls with the end-to-end latency, and of how quickly, were not confirmed. Python's rate there was 3.25 and 4.25 times Java's on the x86 and Neoverse N2 pairs, above the registered limit of 1.5 (Supplement S6). The constants are those of one EEVDF-era kernel. Designs that pin a busy-polling thread to a dedicated core, or timestamp in hardware, remove the waiting state, and we make no claim about them. We read the kernel's timestamps from packet captures and, through a Python module, on the clients' own sockets, at one publish rate on one machine pair and kernel, with the producer and the consumer on one machine. On that virtual network the kernel gave the sockets a departing packet's

timestamp only at the device queue. The clients used neither encryption nor compression, either of which would hide each message's bytes from a module beneath the client library, and we did not measure what taking the timestamps costs a tool.

The deletion is demonstrated in one benchmark, in its embedded mode, and reproduced in one independent program of our own. The benchmark's distributed mode failed on every attempt on the Oracle machines but ran unmodified on the Azure machines. There a moved clock showed its filter dropping values below zero, but nothing counted what it dropped (Supplement S4.5). One registered detail failed: the 32 KB configuration pins at 69.2% retention, not the predicted 66.67%. The audit found the deletion's signature in few public results, so we claim it for those and our own runs only. We classified the 27 tools by reading them, and ran the 32 tools on one pair of machines, one kernel and one publish rate. How much a deployment loses depends on its end-to-end latency and publish rate, so we recommend reporting retention rather than calling any reported number wrong.

IX. Conclusion

A latency benchmark measures the system only while its readings are taken faster than the latency it measures. On one clock, the producer's reading of an acknowledgment and the consumer's reading of the message, which should coincide, differ by milliseconds and in either order, because a thread waits for a core, for up to a scheduler's slice, before it reads the clock. Across the industry such values are deleted, replaced or never measured, a positivity filter on a millisecond clock deletes the fastest messages too, and no report we audited says how many samples it kept. A latency timed between the kernel's timestamps, as each message leaves the client machine and returns, holds no client thread's wait. Under load it held still while, at normal priority, the end-to-end latency's tail rose 4.3–4.6-fold. On Linux, any tool can read those timestamps on its own sockets and report that latency beside the end-to-end latency its application sees. Before acting on a broker latency below a few milliseconds, ask which timestamp it was timed from, how many samples the tool kept, and how many it discarded, by sign.

Acknowledgment

The authors thank N. Herbst, for the load-generation point that became one of the checks above, for pointing us to the timer study cited in Section VII, and for a reading that reset the abstract.

In accordance with IEEE policy, Anthropic's Claude assisted with code development and verification, manuscript editing, and literature search. Experimental design, scientific claims, and interpretation are the authors'.

Artifact Availability

Code, analysis and manuscript are archived at 10.5281/zenodo.21650031, the measurement dataset at 10.5281/zenodo.21650064; both resolve to the current version, `v4.0.0`, whose files are restricted pending publication. Every number here is recomputed from the committed data by the archived code at build time, and the test suite fails if the text and the data disagree.

References

[1] P. Dobbelaere and K. S. Esmaili, "Kafka versus RabbitMQ: A comparative study of two industry reference publish/subscribe implementations," in *Proc. 11th ACM Int. Conf. Distributed and Event-Based Systems (DEBS)*. ACM, 2017, pp. 227–238.

[2] Index.dev, "NATS vs Redis vs Kafka: message broker comparison," Practitioner comparison, 2026, https://www.index.dev/skill-vs-skill/nats-vs-redis-vs-kafka. Accessed: Sep. 3, 2026.

[3] J. Karimov, T. Rabl, A. Katsifodimos, R. Samarev, H. Heiskanen, and V. Markl, "Benchmarking distributed stream data processing systems," in *Proc. IEEE 34th Int. Conf. Data Engineering (ICDE)*, 2018, pp. 1507–1518.

[4] OpenMessaging Project, Linux Foundation, "The OpenMessaging benchmark framework," 2018, available at openmessaging.cloud/docs/benchmarks/. Accessed: Aug. 19, 2026.

[5] I. Stoica, H. Abdel-Wahab, K. Jeffay, S. K. Baruah, J. E. Gehrke, and C. G. Plaxton, "A proportional share resource allocation algorithm for real-time, time-shared systems," in *Proc. 17th IEEE Real-Time Systems Symp. (RTSS)*, 1996, pp. 288–299.

[6] StatsBomb, "Statsbomb open data," *GitHub Repository*, 2023. [Online]. Available: https://github.com/statsbomb/open-data

[7] B. A. Nosek, C. R. Ebersole, A. C. DeHaven, and D. T. Mellor, "The preregistration revolution," *Proc. Natl. Acad. Sci. USA*, vol. 115, no. 11, pp. 2600–2606, 2018.

[8] OpenMessaging Benchmark contributors, "Can the average publish latency be larger than end-to-end latency? (issue #216)," https://github.com/openmessaging/benchmark/issues/216, 2021.

[9] D. Bristot de Oliveira, D. Casini, J. Lelli, and T. Cucinotta, "Timerlat: Real-time Linux scheduling latency measurements, tracing, and analysis," *IEEE Trans. Comput.*, vol. 74, no. 8, pp. 2608–2620, 2025.

[10] B. Gregg, "Linux bcc/BPF run queue (scheduler) latency," https://www.brendangregg.com/blog/2016-10-08/linux-bcc-runqlat.html, Oct. 2016, accessed: Sep. 13, 2026.

[11] Z. Zhou, "sched: Reduce the default slice to avoid tasks getting an extra tick," Linux kernel mailing list, commit 2ae891b82695, 2025.

[12] Broadcom, "RabbitMQ PerfTest — `TimestampProvider.java`," Source repository, 2026, https://github.com/rabbitmq/rabbitmq-perf-test. Accessed: Oct. 2, 2026.

[13] EMQ Technologies, "`emqtt-bench`: MQTT benchmark tool — `src/emqtt_bench.erl`," Source repository, 2026, https://github.com/emqx/emqtt-bench. Accessed: Aug. 21, 2026.

[14] Apache Software Foundation, "Apache Kafka tools — `EndToEndLatency.java` and `ProducerPerformance.java`," Source repository, 2026, https://github.com/apache/kafka. Accessed: Aug. 31, 2026.

[15] IOP Systems, "Rezolus: systems performance telemetry — `scheduler_discarded_samples`," Source repository, 2026, https://github.com/iopsystems/rezolus. Accessed: Aug. 20, 2026.

[16] S. Guo, "Avoid adding negative metric values (pull request #56)," OpenMessaging Benchmark, commit ebb5d6b8, Mar. 2018, https://github.com/openmessaging/benchmark/pull/56. Accessed: Aug. 18, 2026.

[17] Hewlett-Packard, "Time interval averaging," Hewlett-Packard Company, Application Note 162-1, 1970, part no. 02-5952-0766.

[18] J. Corbet, "Completing the EEVDF scheduler," LWN.net, Apr. 2024, https://lwn.net/Articles/969062/. Accessed: Oct. 6, 2026.

[19] J. Li, N. K. Sharma, D. R. K. Ports, and S. D. Gribble, "Tales of the tail: Hardware, OS, and application-level sources of tail latency," in *Proc. ACM SoCC*. ACM, 2014, pp. 1–14.

[20] J. Leverich and C. Kozyrakis, "Reconciling high server utilization and sub-millisecond quality-of-service," in *Proc. 9th European Conf. Computer Systems (EuroSys)*, 2014, pp. 1–14.

[21] D. Bristot de Oliveira, D. Casini, and T. Cucinotta, "Operating system noise in the Linux kernel," *IEEE Trans. Comput.*, vol. 72, no. 1, pp. 196–207, 2023.

[22] Y. Jia, K. Tian, Y. You, Y. Chen, and K. Chen, "Skyloft: A general high-efficient scheduling framework in user space," in *Proc. 30th ACM Symp. Operating Systems Principles (SOSP)*, 2024, pp. 265–279.

[23] M. Kuperberg, M. Krogmann, and R. H. Reussner, "Metric-based selection of timer methods for accurate measurements," in *Proc. 2nd ACM/SPEC Int. Conf. Performance Engineering (ICPE)*, Mar. 2011, pp. 151–156.

[24] Y. Zhang, D. Meisner, J. Mars, and L. Tang, "Treadmill: Attributing the source of tail latency through precise load testing and statistical inference," in *Proc. 43rd ACM/IEEE Int. Symp. Computer Architecture (ISCA)*, 2016, pp. 456–468.

[25] M. Kogias, S. Mallon, and E. Bugnion, "Lancet: A self-correcting latency measuring tool," in *Proc. USENIX Annual Technical Conf. (ATC)*, 2019, pp. 881–896.

[26] A. Chandrasekar and J. Kramberger, "Identifying and mitigating systemic measurement bias in production LLM inference benchmarks," *arXiv preprint arXiv:2605.24217*, 2026.

[27] B. Villain, M. Davis, J. Ridoux, D. Veitch, and N. Normand, "Probing the latencies of software timestamping," in *Proc. IEEE Int. Symp. Precision Clock Synchronization for Measurement, Control and Communication (ISPCS)*, 2012, pp. 1–6.

[28] A. Sharma, D. Shah, D. Lariviere, and H. ElBakoury, "Time, causality, and observability failures in distributed AI inference systems," *arXiv preprint arXiv:2604.21361*, Apr. 2026, Open Compute Project, Unified Intelligent Infrastructure workstream.

[29] G. Van Dongen and D. Van den Poel, "Evaluation of stream processing frameworks," *IEEE Trans. Parallel Distrib. Syst.*, vol. 31, no. 8, pp. 1845–1858, 2020.

[30] IEEE, "IEEE standard for a precision clock synchronization protocol for networked measurement and control systems," IEEE Std 1588-2019, 2020, revision of IEEE Std 1588-2008.

[31] S. Yang, J. Jeong, B. Scholz, and B. Burgstaller, "Cloudprofiler: TSC-based inter-node profiling and high-throughput data ingestion for cloud streaming workloads," *arXiv preprint arXiv:2205.09325*, 2022.

[32] R. Kundel, "P4STA: High precision network function benchmarking," in *Accelerating Network Functions Using Reconfigurable Hardware*, ser. Springer Theses. Springer Nature Switzerland, 2024, pp. 93–115.

[33] P. B. Danzig and S. Melvin, "High resolution timing with low resolution clocks and a microsecond resolution timer for sun workstations," *ACM SIGOPS Operating Systems Review*, vol. 24, no. 1, pp. 23–26, Jan. 1990.

[34] V. Paxson, G. Almes, J. Mahdavi, and M. Mathis, "Framework for IP performance metrics," Internet Engineering Task Force, RFC 2330, May 1998.

[35] V. Raisanen, G. Grotefeld, and A. Morton, "Network performance measurement with periodic streams," Internet Engineering Task Force, RFC 3432, Nov. 2002.

[36] F. Baccelli, S. Machiraju, D. Veitch, and J. C. Bolot, "The role of PASTA in network measurement," in *Proc. ACM SIGCOMM*, 2006, pp. 231–242.

[37] G. Tene, "How NOT to measure latency," 2015, talk, Strange Loop 2015, St. Louis, MO, USA. https://www.youtube.com/watch?v=lJ8ydIuPFeU.

[38] V. Paxson, "On calibrating measurements of packet transit times," in *Proc. ACM SIGMETRICS*. ACM, 1998, pp. 11–21.

[39] S. Shalunov, B. Teitelbaum, A. Karp, J. W. Boote, and M. J. Zekauskas, "A one-way active measurement protocol (OWAMP)," Internet Engineering Task Force, RFC 4656, Sep. 2006.

[40] T. Mytkowicz, A. Diwan, M. Hauswirth, and P. F. Sweeney, "Producing wrong data without doing anything obviously wrong!" in *Proc. 14th Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 2009, pp. 265–276.

[41] A. Georges, D. Buytaert, and L. Eeckhout, "Statistically rigorous Java performance evaluation," in *Proc. 22nd ACM SIGPLAN Conf. Object-Oriented Programming, Systems, Languages and Applications (OOPSLA)*. ACM, 2007, pp. 57–76.

[42] T. Hoefler and R. Belli, "Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results," in *Proc. Int. Conf. High Performance Computing, Networking, Storage and Analysis (SC)*. ACM, 2015, pp. 73:1–73:12.

[43] A. V. Papadopoulos, L. Versluis, A. Bauer, N. Herbst, J. von Kistowski, A. Ali-Eldin, C. L. Abad, J. N. Amaral, P. Tůma, and A. Iosup, "Methodological principles for reproducible performance evaluation in cloud computing," *IEEE Trans. Softw. Eng.*, vol. 47, no. 8, pp. 1528–1543, 2021.

[44] S. Henning, A. Vogel, M. Leichtfried, O. Ertl, and R. Rabiser, "ShuffleBench: A benchmark for large-scale data shuffling operations with distributed stream processing frameworks," in *Proc. 15th ACM/SPEC Int. Conf. Performance Engineering (ICPE)*, 2024, pp. 2–13.

[45] Linux Kernel Documentation, "KVM-specific MSRs: `MSR_KVM_SYSTEM_TIME_NEW`," https://docs.kernel.org/virt/kvm/x86/msr.html, 2026, accessed: Aug. 21, 2026.

Gustavo Pedro Ricou received the B.Sc. degree in physics from the Universidade do Porto, Portugal, and specialized in nuclear fusion science at the Universität Stuttgart, Germany, and the Université de Lorraine, France. He holds an M.Sc. in software design with cloud-native computing from the Technological University of the Shannon, with first-class honors, and is currently pursuing an M.Sc. in statistics and data science at Trinity College Dublin, Ireland. He is a Software Engineer with BoscaSports. His research concerns measurement validity — what a reported number is really evidence of — in computer systems, environmental measurement and sports analytics, using pre-registration and physical-consistency auditing.

Romaric Duvignau received the Ph.D. degree in computer science from the University of Bordeaux, France, in 2015, for work on the maintenance and simulation of dynamic random graphs. He is an Associate Professor and Head of the Network and Systems unit at Chalmers University of Technology, Gothenburg, Sweden. His research concerns distributed and data-driven algorithms, the monitoring of large-scale distributed systems, and stream analytics for cyber-physical systems.